

JAVA BANK MANAGEMENT SYSTEM

Object-Oriented Programming Practical Record

Question - Sample Case Study on Classes & Objects (CO1)

Bank Account Management System

Problem Statement: Design a system to manage bank accounts.

Classes: Account, Savings-Account, Current-Account.

Attributes: Account number, balance, account type, interest rate (for savings account), overdraft limit (for current account), etc.

Methods: Deposit, Withdraw, Calculate Interest, Transfer, etc.

Object Usage: Create objects for each customer's account and perform transactions such as deposits, withdrawals, and transfers.

Aim: To implement this menu-driven system in Java using inheritance, method overriding, encapsulated account operations, and polymorphism.

Submitted by: G. Pranadeep

Roll No.: A25126552212

Class: 2/4 CSM-D

Branch: Computer Science (Artificial Intelligence / Machine Learning)

1. Steps for Execution

1. Open any Java development environment such as VS Code, IntelliJ IDEA, Eclipse, or a command prompt with the JDK installed.
2. Create a new Java source file named Bank.java.
3. Copy the complete program from this record into the file and save it.
4. Compile the source file using `javac Bank.java`.
5. Run the compiled program using `java Bank`.
6. Choose an option from the displayed menu, enter the requested account/amount values, and continue until option 6 is selected.

Initial Account Data

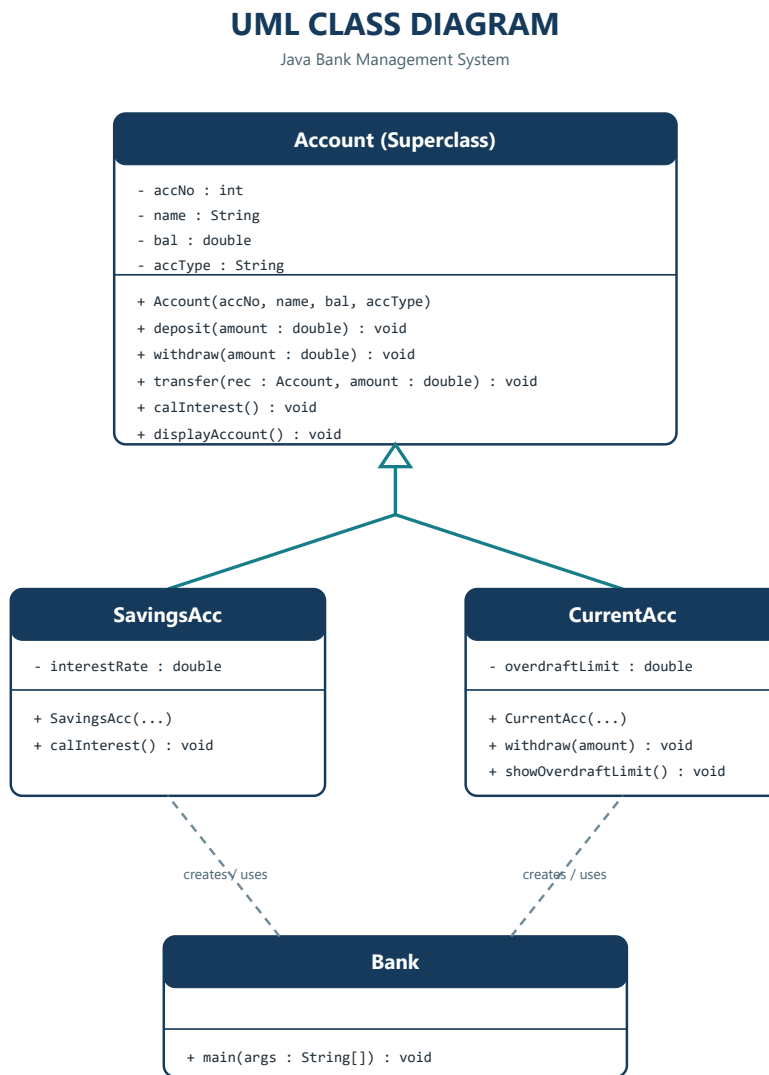
Account	Account No.	Type	Opening Balance	Special Setting
Bunny	1001	Savings	₹10,000	5% interest rate
Arya	2001	Current	₹5,000	₹3,000 overdraft limit

2. Algorithm

1. Start the program and create a Scanner object to receive keyboard input.
2. Create a SavingsAcc object for Bunny and a CurrentAcc object for Arya with their initial values.
3. Display the bank menu and read the user's choice.
4. For Deposit, select an account and add the entered amount only if it is positive.
5. For Withdraw, call the selected account's withdraw method. Savings uses available balance; Current additionally permits withdrawal up to balance plus overdraft limit.
6. For Transfer, select the direction and transfer the amount only if the sending account has sufficient balance.
7. For Calculate Interest, invoke `calInterest()`. SavingsAcc overrides this method to calculate $\text{balance} \times \text{interest rate} / 100$; Account displays that interest is not applicable.
8. For Display Accounts, print both account details and Arya's overdraft limit.
9. Repeat the menu until choice 6 is selected, then display the exit message and stop.

3. UML Class Diagram

The diagram shows inheritance from Account and the Bank class that creates and uses the account objects.



SavingsAcc and CurrentAcc inherit from Account

Key OOP Concepts: inheritance (SavingsAcc and CurrentAcc extend Account), method overriding (calInterest and withdraw), and polymorphism through common Account operations.

4. Java Source Code

Filename: Bank.java. The following is the original program logic with explanatory comments added only.

```
import java.util.Scanner;

// Base class that stores common account data and operations.
class Account {
    int accNo;
    String name;
    double bal;
    String accType;

    public Account(int accNo, String name, double bal, String accType) {
        this.accNo = accNo;
        this.name = name;
        this.bal = bal;
        this.accType = accType;
    }

    // Adds a positive amount to the balance.
    public void deposit(double amount) {
        if (amount > 0) {
            bal += amount;
            System.out.println("₹" + amount + " deposited successfully.");
        } else {
            System.out.println("Invalid deposit amount.");
        }
    }

    // Regular accounts cannot withdraw more than their balance.
    public void withdraw(double amount) {
        if (amount <= bal) {
            bal -= amount;
            System.out.println("₹" + amount + " withdrawn successfully.");
        } else {
            System.out.println("Insufficient Balance!");
        }
    }

    // Transfers money when the sender has sufficient balance.
    public void transfer(Account rec, double amount) {
        if (amount <= bal) {
            bal -= amount;
            rec.bal += amount;
            System.out.println("₹" + amount + " transferred to " + rec.name);
        } else {
            System.out.println("Transfer Failed! Insufficient Balance.");
        }
    }

    public void calInterest() {
        System.out.println("Interest calculation not applicable.");
    }

    // Displays the details of the current account object.
    public void displayAccount() {
        System.out.println("\n-----");
        System.out.println("Account Number : " + accNo);
        System.out.println("Account Holder : " + name);
        System.out.println("Account Type : " + accType);
        System.out.println("Balance : ₹" + bal);
    }
}

// Savings account calculates interest using its interest rate.
class SavingsAcc extends Account {
    double interestRate;

    public SavingsAcc(int accNo, String name, double bal, double interestRate) {
        super(accNo, name, bal, "Savings");
        this.interestRate = interestRate;
    }

    @Override
    public void calInterest() {
        double interest = bal * interestRate / 100;
        System.out.println("Interest Earned : ₹" + interest);
    }
}
```

4. Java Source Code (continued)

```
// Current account can use an overdraft limit when withdrawing.
class CurrentAcc extends Account {
    double overdraftLimit;

    public CurrentAcc(int accNo, String name, double bal, double overdraftLimit) {
        super(accNo, name, bal, "Current");
        this.overdraftLimit = overdraftLimit;
    }

    @Override
    public void withdraw(double amount) {
        if (bal + overdraftLimit >= amount) {
            bal -= amount;
            System.out.println("₹" + amount + " withdrawn successfully.");
        } else {
            System.out.println("Overdraft Limit Exceeded!");
        }
    }

    public void showOverdraftLimit() {
        System.out.println("Overdraft Limit : ₹" + overdraftLimit);
    }
}

public class Bank {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        SavingsAcc customer1 = new SavingsAcc(1001, "Bunny", 10000, 5);
        CurrentAcc customer2 = new CurrentAcc(2001, "Arya", 5000, 3000);
        int choice;

        // Repeats the menu until the user chooses Exit.
        do {
            System.out.println("\n===== BANK MENU =====");
            System.out.println("1. Deposit");
            System.out.println("2. Withdraw");
            System.out.println("3. Transfer");
            System.out.println("4. Calculate Interest");
            System.out.println("5. Display Accounts");
            System.out.println("6. Exit");
            System.out.print("Enter your choice: ");
            choice = sc.nextInt();

            switch (choice) {
                case 1:
                    System.out.println("Select Account:");
                    System.out.println("1. Bunny (Savings)");
                    System.out.println("2. Arya (Current)");
                    int dAcc = sc.nextInt();
                    System.out.print("Enter amount: ");
                    double dAmt = sc.nextDouble();
                    if (dAcc == 1)
                        customer1.deposit(dAmt);
                    else if (dAcc == 2)
                        customer2.deposit(dAmt);
                    else
                        System.out.println("Invalid Account!");
                    break;
                case 2:
                    System.out.println("Select Account:");
                    System.out.println("1. Bunny (Savings)");
                    System.out.println("2. Arya (Current)");
                    int wAcc = sc.nextInt();
                    System.out.print("Enter amount: ");
                    double wAmt = sc.nextDouble();
                    if (wAcc == 1)
                        customer1.withdraw(wAmt);
                    else if (wAcc == 2)
                        customer2.withdraw(wAmt);
                    else
                        System.out.println("Invalid Account!");
                    break;
            }
        } while (choice != 6);
    }
}
```

4. Java Source Code (continued)

```
        case 3:
            System.out.println("Transfer:");
            System.out.println("1. Bunny -> Arya");
            System.out.println("2. Arya -> Bunny");
            int t = sc.nextInt();
            System.out.print("Enter amount: ");
            double tAmt = sc.nextDouble();
            if (t == 1)
                customer1.transfer(customer2, tAmt);
            else if (t == 2)
                customer2.transfer(customer1, tAmt);
            else
                System.out.println("Invalid Choice!");
            break;
        case 4:
            System.out.println("1. Bunny (Savings)");
            System.out.println("2. Arya (Current)");
            int i = sc.nextInt();
            if (i == 1)
                customer1.calInterest();
            else if (i == 2)
                customer2.calInterest();
            else
                System.out.println("Invalid Choice!");
            break;
        case 5:
            customer1.displayAccount();
            customer2.displayAccount();
            customer2.showOverdraftLimit();
            break;
        case 6:
            System.out.println("Thank you for using Bank Management System.");
            break;
        default:
            System.out.println("Invalid Choice!");
    }
} while (choice != 6);
}
```

5. Representative Sample Output

The run below uses these inputs in sequence: deposit ₹2,000 into Bunny; withdraw ₹7,000 from Arya; transfer ₹1,500 from Bunny to Arya; calculate Bunny's interest; display accounts; exit. Therefore Bunny's final balance is ₹10,500 and Arya's final balance is ₹-500.

```

===== BANK MENU =====
1. Deposit
2. Withdraw
3. Transfer
4. Calculate Interest
5. Display Accounts
6. Exit
Enter your choice: 1
Select Account:
1. Bunny (Savings)
2. Arya (Current)
1
Enter amount: 2000
₹2000.0 deposited successfully.

===== BANK MENU =====
Enter your choice: 2
Select Account:
1. Bunny (Savings)
2. Arya (Current)
2
Enter amount: 7000
₹7000.0 withdrawn successfully.

===== BANK MENU =====
Enter your choice: 3
Transfer:
1. Bunny -> Arya
2. Arya -> Bunny
1
Enter amount: 1500
₹1500.0 transferred to Arya

===== BANK MENU =====
Enter your choice: 4
1. Bunny (Savings)
2. Arya (Current)
1
Interest Earned : ₹525.0

===== BANK MENU =====
Enter your choice: 5

-----
Account Number : 1001
Account Holder : Bunny
Account Type   : Savings
Balance       : ₹10500.0

-----
Account Number : 2001
Account Holder : Arya
Account Type   : Current
Balance       : ₹-500.0
Overdraft Limit : ₹3000.0

===== BANK MENU =====
Enter your choice: 6
Thank you for using Bank Management System.

```

Result: The menu-driven Java Bank Management System was executed successfully. It demonstrates account management, inheritance, method overriding, interest calculation, transfers, and current-account overdraft handling.